

Homework — 1.2.4 Types of Programming Language — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. [3] Programming paradigm = **B**; Procedural language = **F**; Assembly language = **D**; Class = **E**; Inheritance = **A**; Encapsulation = **C**. (2 correct matches per mark.)

2 (a) [3] Completed trace — the branch is **not** taken because the ACC is negative ($5 - 9 = -4$):

Instruction executed	ACC	first	Output
INP	9		
STA first	9	9	
INP	5	9	
SUB first	-4	9	
BRP big	-4	9	
LDA first	9	9	
OUT	9	9	9
HLT	9	9	

1 mark: instruction column correct — BRP big is followed by LDA first, OUT, HLT (branch not taken). 1 mark: ACC column 9, 5, -4, -4, 9, 9, 9. 1 mark: first column shows 9 from STA first onwards **and** output **9** on the OUT row. (Accept unchanged cells left blank.)

2 (b) [1] It outputs the **larger** of the two numbers input (accept: if they are equal it outputs that value) (1).

3. [4] 1 mark per row: Immediate = **4** — the operand itself is the value; Direct = **7** — the value held at address 4; Indirect = **2** — address 4 holds 7, which is used as the address, and address 7 holds 2; Indexed = **9** — effective address = operand 4 + index register 2 = address 6, which holds 9.

4. [3] Up to 3 from: common behaviour (moving, holding items) is defined once in a superclass such as Character, and each character type is a class (1); the wizard/archer/guard subclasses **inherit** this behaviour and add or override their own, so code is not duplicated and the game is easier to maintain and extend with new character types (1); **encapsulation** keeps each object's attributes (e.g. health, position) private, accessible only through its methods, so other parts of the program cannot put a character into an invalid state (1); classes/objects map naturally onto a game made of many similar interacting entities (1). Max 3.

Part B (6 marks)

5. [3] 1 mark per correct row:

- Row 1: **True**.
- Row 2: **False** — in Waterfall the client normally only sees the product at/near the **end** of the project (early, frequent delivery of working software is a feature of Agile).
- Row 3: **False** — extreme programming is an example of an **Agile** methodology.

6. [2] Static linking copies the required library code into the executable at compile/link time, so the executable is self-contained but larger, and library updates require re-linking (1). Dynamic linking keeps the library in a separate file (e.g. a DLL) that is linked/loaded at run time, so the executable is smaller and the library can be shared and updated independently (1).

7. [1] The Accumulator temporarily holds the data being processed / the results of arithmetic and logic operations carried out by the ALU (1).

Total: 14 + 6 = 20 marks